Software Engineering

CE2006/CZ2006

Agile Software Development

Topics covered

- ✧ Agile methods
- ✧ Agile development techniques
- ✧ Agile project management

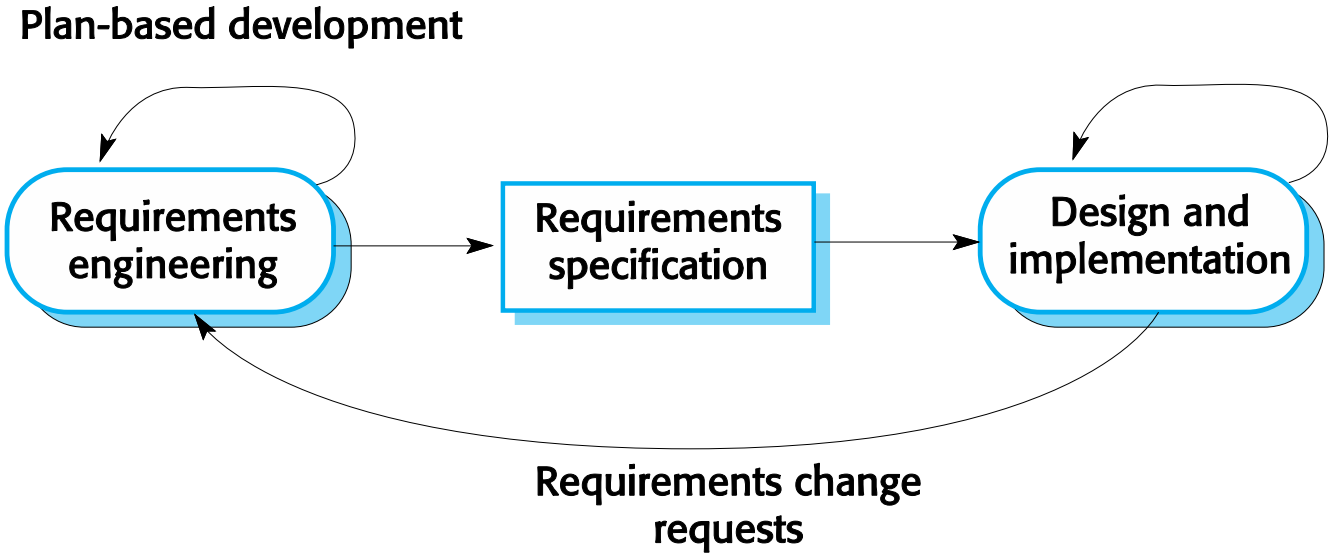
Rapid software development

- Rapid development and delivery is now often the most important requirement for software systems
  - Businesses operate in a fast –changing requirement and it is practically impossible to produce a set of stable software requirements
  - Software has to evolve quickly to reflect changing business needs.
- Plan-driven development is essential for some types of system but does not meet these business needs.
- Agile development methods emerged in the late 1990s whose aim was to radically reduce the delivery time for working software systems

Agile development

- Program specification, design and implementation are inter-leaved
- The system is developed as a series of versions or increments with stakeholders involved in version specification and evaluation
- Frequent delivery of new versions for evaluation
- Extensive tool support (e.g. automated testing tools) used to support development.
- Minimal documentation – focus on working code

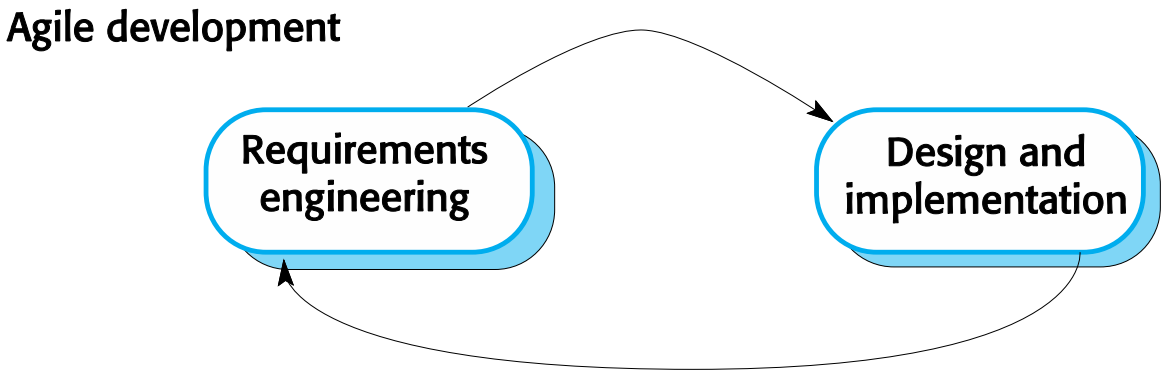
Plan-driven and agile development



✧ Plan-driven development

- A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance.
- Not necessarily waterfall model – plan-driven, incremental development is possible
- Iteration occurs within activities.

Plan-driven and agile development



- Agile development
  - Specification, design, implementation and testing are inter-leaved and the outputs from the development process are decided through a process of negotiation during the software development process.

Agile methods

Agile manifesto

- The aim of agile methods is to reduce overheads in the software process (e.g. by limiting documentation) and to be able to respond quickly to changing requirements without excessive rework.
- *We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:*
  - *Individuals and interactions over processes and tools*
    Working software over comprehensive documentation
    Customer collaboration over contract negotiation
    Responding to change over following a plan
- *That is, while there is value in the items on the right, we value the items on the left more.*

The principles of agile methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

Agile method applicability

- Product development where a software company is developing a small or medium-sized product for sale.
- Custom system development within an organization, where there is a clear commitment from the customer to become involved in the development process and where there are few external rules and regulations that affect the software.

Agile methods across organizations

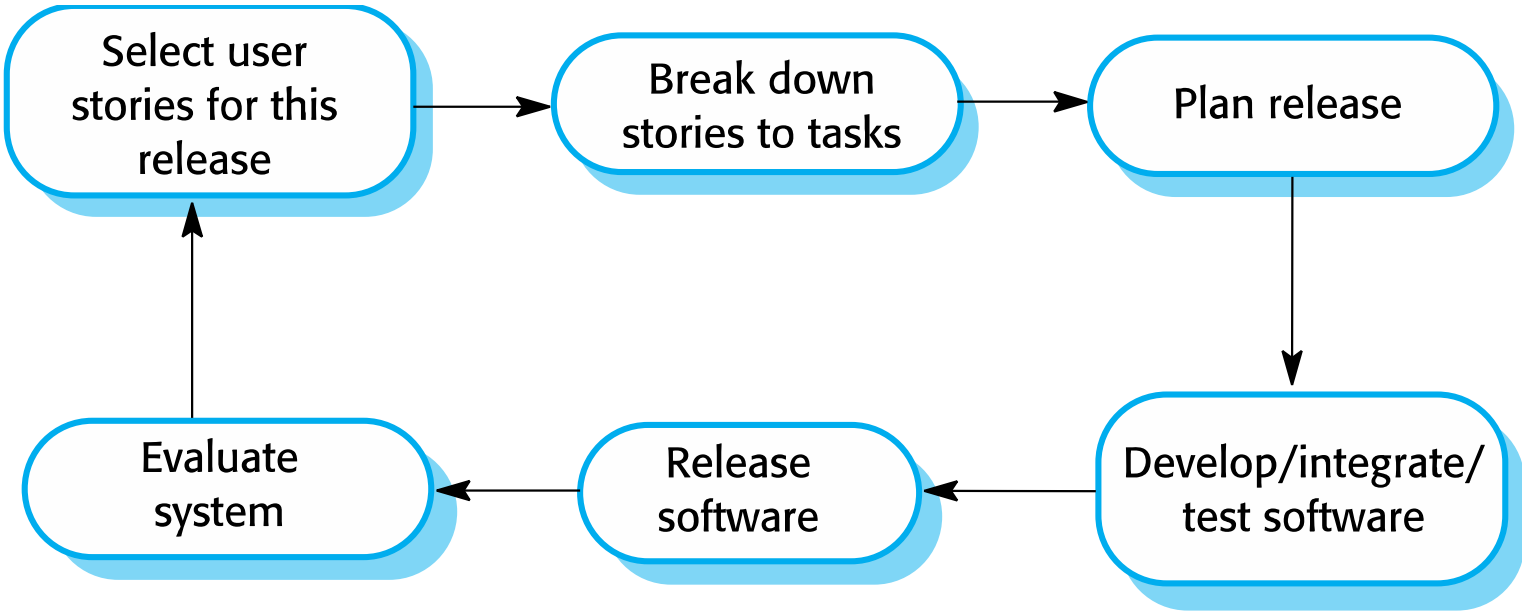
- Project managers who do not have experience of agile methods may be reluctant to accept the risk of a new approach.
- Large organizations often have quality procedures and standards that all projects are expected to follow and, because of their bureaucratic nature, these are likely to be incompatible with agile methods.
- Agile methods seem to work best when team members have a relatively high skill level. However, within large organizations, there are likely to be a wide range of skills and abilities.
- There may be cultural resistance to agile methods, especially in those organizations that have a long history of using conventional systems engineering processes.

Agile development techniques

Extreme programming

- A very influential agile method, developed in the late 1990s, that introduced a range of agile development techniques.
- Extreme Programming (XP) takes an ‘extreme’ approach to iterative development.
  - New versions may be built several times per day;
  - Increments are delivered to customers every 2 weeks;
  - All tests must be run for every build and the build is only accepted if tests run successfully.

The extreme programming release cycle



Extreme programming practices (a)

Principle or practice	Description
Incremental planning	Requirements are recorded on story cards and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development 'Tasks'. See Figures 3.5 and 3.6.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Simple design	Enough design is carried out to meet the current requirements and no more.
Test-first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.

Extreme programming practices (b)

Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Sustainable pace	Large amounts of overtime are not considered acceptable as the net effect is often to reduce code quality and medium term productivity
On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

Influential XP practices

- Extreme programming has a technical focus and is not easy to integrate with management practice in most organizations.
- Consequently, while agile development uses practices from XP, the method as originally defined is not widely used.
- Key practices
  - User stories for specification
  - Refactoring
  - Test-first development
  - Pair programming

User stories for requirements

- In XP, a customer or user is part of the XP team and is responsible for making decisions on requirements.
- User requirements are expressed as user stories or scenarios.
- These are written on cards and the development team break them down into implementation tasks. These tasks are the basis of schedule and cost estimates.
- The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.

A ‘prescribing medication’ story

Prescribing medication

The record of the patient must be open for input. Click on the medication field and select either ‘current medication’, ‘new medication’ or ‘formulary’.

If you select ‘current medication’, you will be asked to check the dose; If you wish to change the dose, enter the new dose then confirm the prescription.

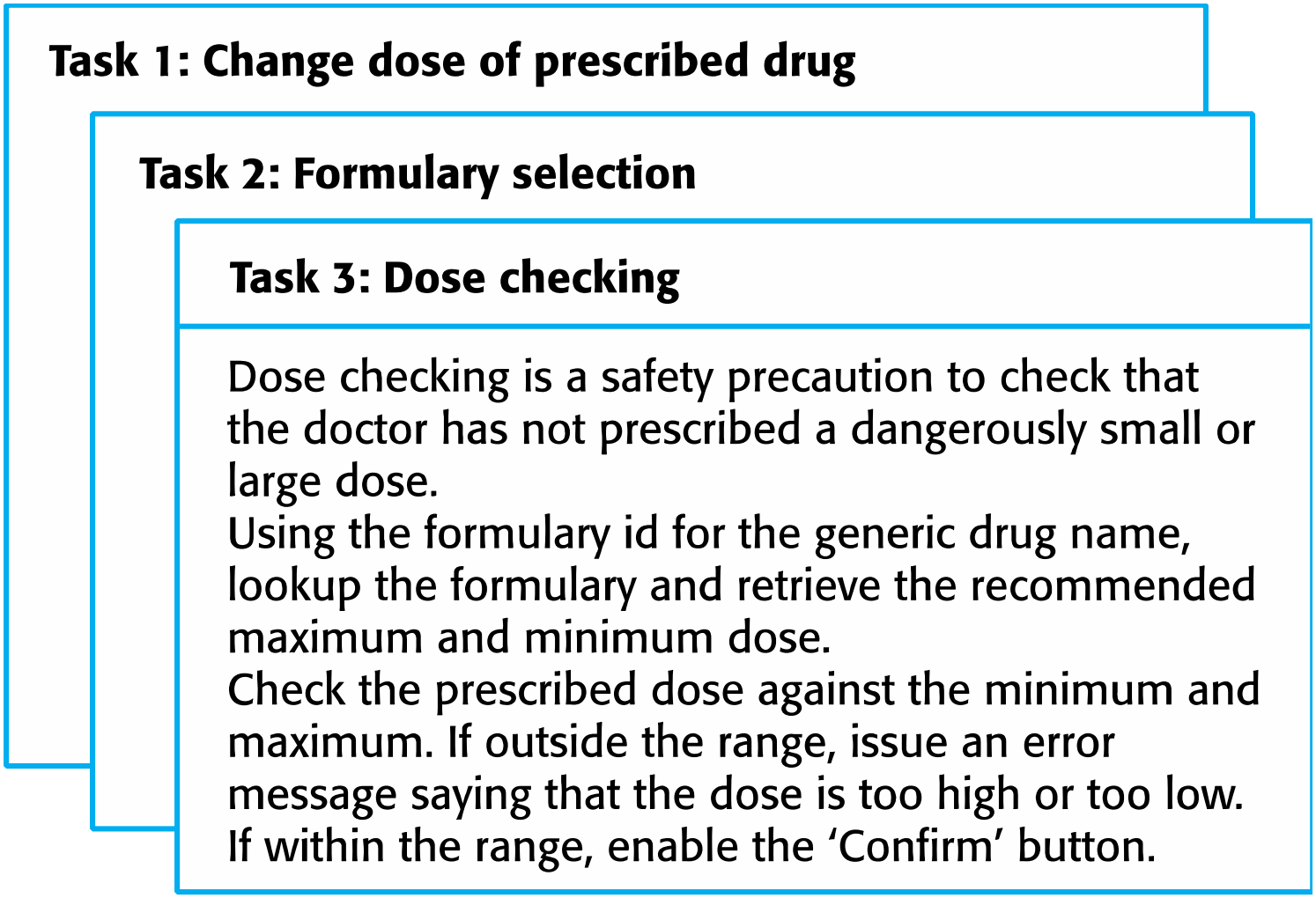
If you choose, ‘new medication’, the system assumes that you know which medication you wish to prescribe. Type the first few letters of the drug name. You will then see a list of possible drugs starting with these letters. Choose the required medication. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

If you choose ‘formulary’, you will be presented with a search box for the approved formulary. Search for the drug required then select it. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

In all cases, the system will check that the dose is within the approved range and will ask you to change it if it is outside the range of recommended doses.

After you have confirmed the prescription, it will be displayed for checking. Either click ‘OK’ or ‘Change’. If you click ‘OK’, your prescription will be recorded on the audit database. If you click ‘Change’, you reenter the ‘Prescribing medication’ process.

Examples of task cards for prescribing medication



Refactoring

- Conventional wisdom in software engineering is to design for change. It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- XP, however, maintains that this is not worthwhile as changes cannot be reliably anticipated.
- Rather, it proposes constant code improvement (refactoring) to make changes easier when they have to be implemented.

Refactoring

- Programming team look for possible software improvements and make these improvements even where there is no immediate need for them.
- This improves the understandability of the software and so reduces the need for documentation.
- Changes are easier to make because the code is well-structured and clear.
- However, some changes requires architecture refactoring and this is much more expensive.

Examples of refactoring

- Re-organization of a class hierarchy to remove duplicate code.
- Tidying up and renaming attributes and methods to make them easier to understand.
- The replacement of inline code with calls to methods that have been included in a program library.

Test-first development

- Testing is central to XP and XP has developed an approach where the program is tested after every change has been made.
- XP testing features:
  - Test-first development.
  - Incremental test development from scenarios.
  - User involvement in test development and validation.
  - Automated test harnesses are used to run all component tests each time that a new release is built.

Test-driven development

- Writing tests before code clarifies the requirements to be implemented.
- Tests are written as programs rather than data so that they can be executed automatically. The test includes a check that it has executed correctly.
  - Usually relies on a testing framework such as Junit.
- All previous and new tests are run automatically when new functionality is added, thus checking that the new functionality has not introduced errors.

Test automation

- Test automation means that tests are written as executable components before the task is implemented
  - These testing components should be stand-alone, should simulate the submission of input to be tested and should check that the result meets the output specification. An automated test framework (e.g. Junit) is a system that makes it easy to write executable tests and submit a set of tests for execution.
- As testing is automated, there is always a set of tests that can be quickly and easily executed
  - Whenever any functionality is added to the system, the tests can be run and problems that the new code has introduced can be caught immediately.

Pair programming

- In pair programming, programmers sit together at the same computer to develop the software.
- Pairs are created dynamically so that all team members work with each other during the development process.
- The sharing of knowledge that happens during pair programming is very important as it reduces the overall risks to a project when team members leave.
- Pair programming is not necessarily inefficient and there is some evidence that suggests that a pair working together is more efficient than 2 programmers working separately.

Pair programming

- Pair programming involves programmers working in pairs, developing code together.
- This helps develop common ownership of code and spreads knowledge across the team.
- It serves as an informal review process as each line of code is looked at by more than 1 person.
- It encourages refactoring as the whole team can benefit from improving the system code.

Agile project management

Agile project management

- The principal responsibility of software project managers is to manage the project so that the software is delivered on time and within the planned budget for the project.
- The standard approach to project management is plan-driven. Managers draw up a plan for the project showing what should be delivered, when it should be delivered and who will work on the development of the project deliverables.
- Agile project management requires a different approach, which is adapted to incremental development and the practices used in agile methods.

Scrum

- Scrum is an agile method that focuses on managing iterative development rather than specific agile practices.
- There are three phases in Scrum.
  - The initial phase is an outline planning phase where you establish the general objectives for the project and design the software architecture.
  - This is followed by a series of sprint cycles, where each cycle develops an increment of the system.
  - The project closure phase wraps up the project, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project.
-

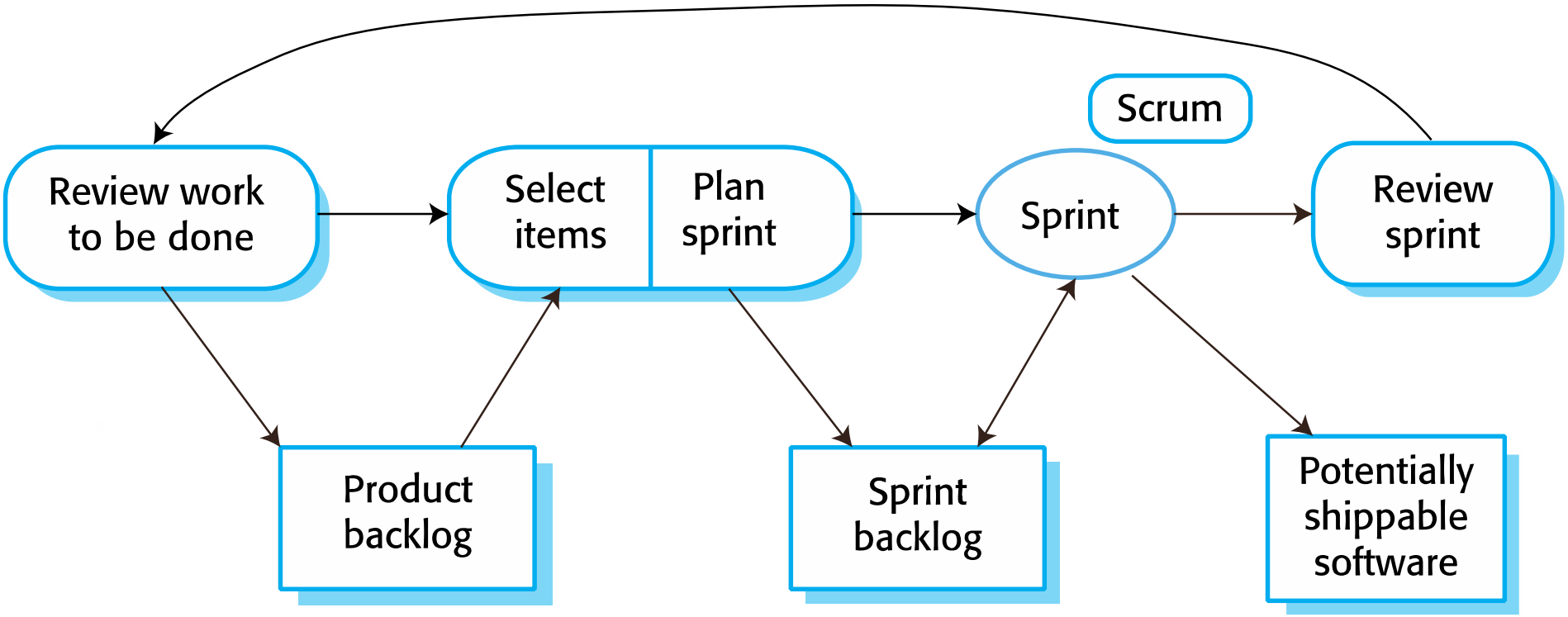
Scrum terminology (a)

Scrum term	Definition
Development team	A self-organizing group of software developers, which should be no more than 7 people. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. The idea is that this should be 'potentially shippable' which means that it is in a finished state and no further work, such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	This is a list of 'to do' items which the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development and continuously review the product backlog to ensure that the project continues to meet critical business needs. The Product Owner can be a customer but might also be a product manager in a software company or other stakeholder representative.

Scrum terminology (b)

Scrum term	Definition
Scrum	A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a short face-to-face meeting that includes the whole team.
ScrumMaster	The ScrumMaster is responsible for ensuring that the Scrum process is followed and guides the team in the effective use of Scrum. He or she is responsible for interfacing with the rest of the company and for ensuring that the Scrum team is not diverted by outside interference. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager. Others, however, may not always find it easy to see the difference.
Sprint	A development iteration. Sprints are usually 2-4 weeks long.
Velocity	An estimate of how much product backlog effort that a team can cover in a single sprint. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

Scrum sprint cycle



The Scrum sprint cycle

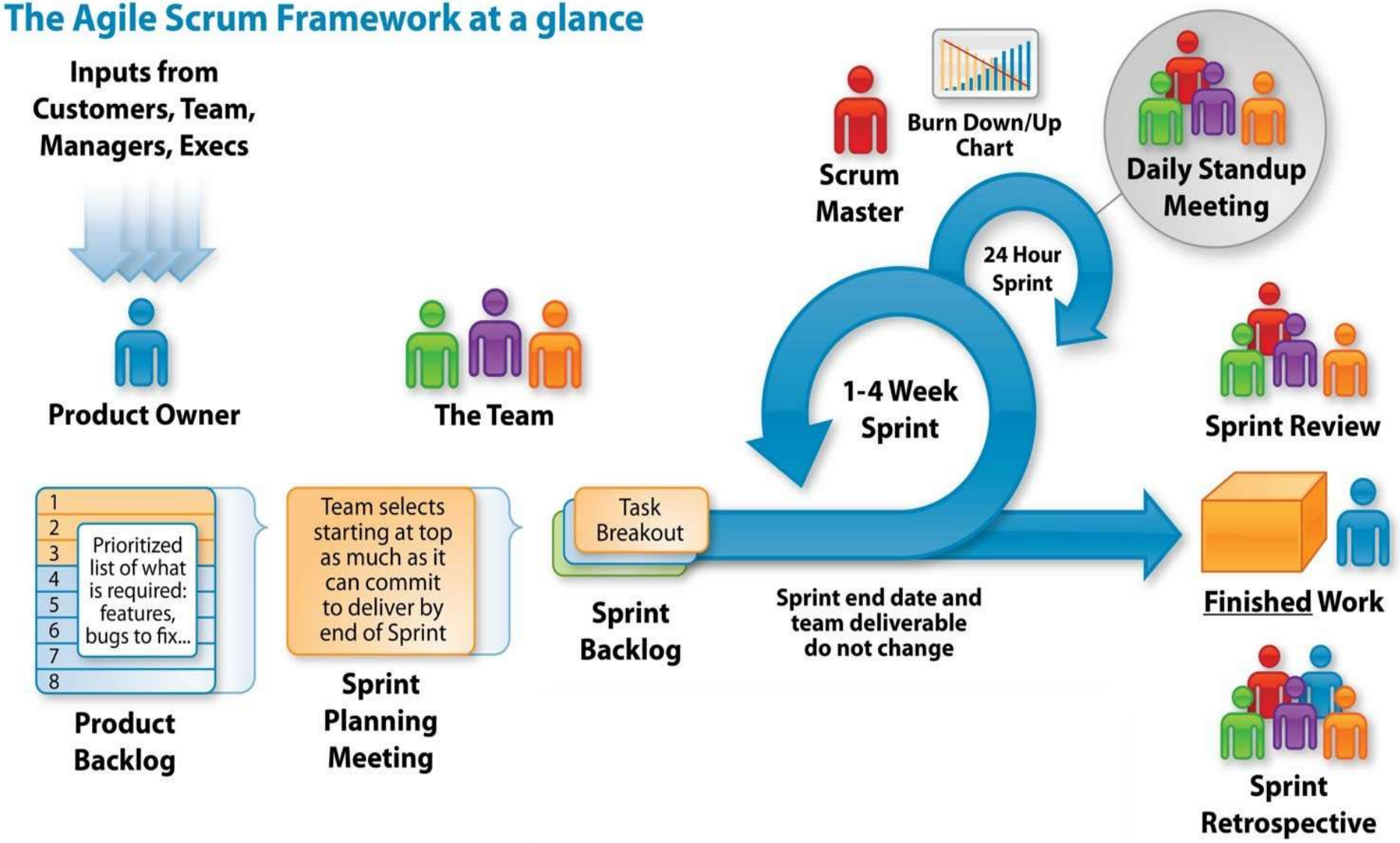
- Sprints are fixed length, normally 2–4 weeks.
- The starting point for planning is the product backlog, which is the list of work to be done on the project.
- The selection phase involves all of the project team who work with the customer to select the features and functionality from the product backlog to be developed during the sprint.

The Sprint cycle

- Once these are agreed, the team organize themselves to develop the software.
- During this stage the team is isolated from the customer and the organization, with all communications channelled through the so-called ‘Scrum master’.
- The role of the Scrum master is to protect the development team from external distractions.
- At the end of the sprint, the work done is reviewed and presented to stakeholders. The next sprint cycle then begins.

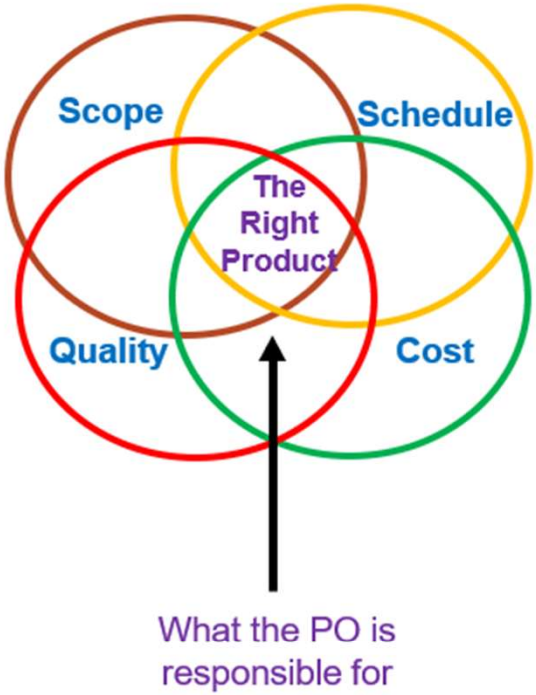
Practical Aspects of Scrum

The Agile Scrum Framework at a glance



SCRUM ROLE – PRODUCT OWNER

- Has a clear vision and goals for the Product. Represents the interests of the customers/stakeholders.
- Responsible for **maximizing the value of the Product** resulting from the work of the Development Team.
- Owns the Product Backlog.
  - Creation
  - Management
  - Prioritization



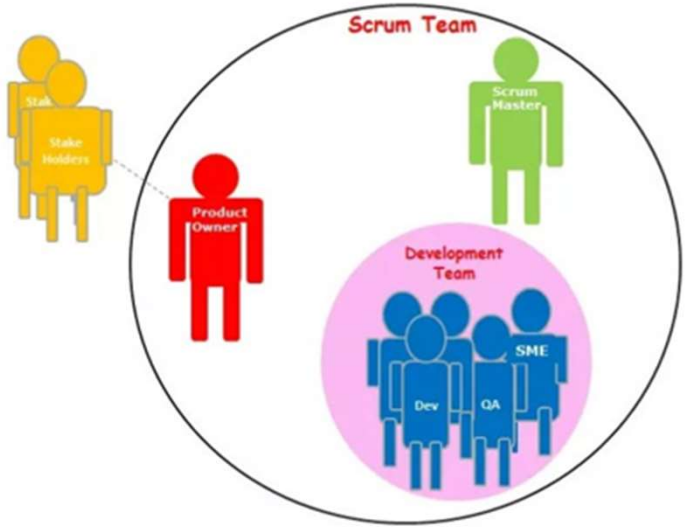
SCRUM ROLE – SCRUM MASTER

- A **servant-leader** - focus is on the needs of the team members and those they serve (the customer), with the goal of achieving results in line with the organization's values, principles and business objectives

- Teaches and coaches the organization and team in adopting and using Scrum process to perform at the highest level.

- Facilitates Scrum events.

- Helps team to remove and prevent **impediments**.



SCRUM ROLE – DEVELOPMENT TEAM

- A team of 5 to 9 professionals who **is responsible for the working product** at the end of each Sprint.
- Flat structure: no titles other than developer, no sub-team.
- Owns Sprint backlog and decides how much is realistic for each Sprint
- Produces thoroughly tested, defect-free “Done” Product Increment in each Sprint.

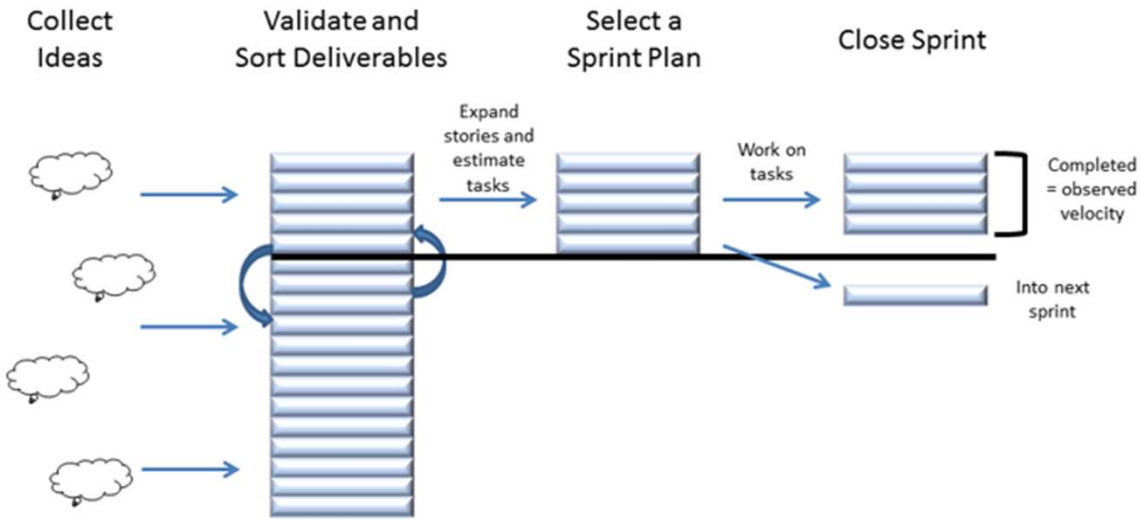
SCRUM ARTIFACT- PRODUCT BACKLOG

- An **ordered list** of everything that is known to be needed in or for the Product to achieved maximum value.
- Is the **single source of requirements** for any changes to be made to the product.
- Product Owner is responsible for it, including its content, availability and ordering
- **User Stories** are a way of expressing **Features** on the Product Backlog.

SCRUM ARTIFACT- SPRINT BACKLOG

- The set of **Product Backlog items (PBIs)** selected for the Sprint, plus a plan for delivering the product Increment and realizing the Sprint Goal.
- The sprint backlog is **unchanged** during the period of the **Sprint**.
- Items **not** "Done" by end of the Sprint - they will be added back to the Product Backlog and addressed during the next Sprint.
- Anything needs improvement - can be put back to the Product Backlog. Either add additional Sprints to the schedule or remove lower priority items from the Product Backlog.

SCRUM ARTIFACT- SPRINT BACKLOG



- PBIs are broken down into tasks.
- Tasks are estimated in hours, usually 1 - 16.Task that more than 16 hours are broken down further later on.
- Team members sign up for task. They are not assigned.
- Estimated remaining work is updated daily.

SCRUM ARTIFACT- PRODUCT INCREMENT

- At end of each Sprint, Done" items will be integrated and tested with the increments of all previous Sprints. -> **Potentially Shippable Product**

- The Product Increment is in a potentially releasable state – Product Owner could release it they wanted to do with **the collection of completed features** (ie: the delivery of a Release)

SCRUM EVENT - SPRINT

- A time-box event of **2 to 4 weeks** when a Scrum Team works to complete a set amount of work in which a "Done" product increment is created.

- Each sprint has a goal of what is to be built. The **Sprint Goal** does not change during the sprint.

- The time-box is never extended. Sprint ends when time box expires.

- At every sprint, the Scrum Team will **inspect** and **adapt** the **Product** and the **Process**.

- ***Can the Sprint be cancelled before its duration is over? If so, who can cancel the Sprint?***

WHAT WORK CAN GET DONE IN THIS SPRINT?

- Product Owner presents the **ordered** product backlog items (PBIs) to the Development team.
- Scrum Master **facilitates**, gives advice, asks question and helps identify risks etc.
- The PBIs are broken down to tasks and tasks are estimated jointly in hours.
- Only the Development Team can assess how much can be accomplished in the Sprint.

HOW WILL THE CHOSEN WORK GET DONE?

- The Development Team decides how it will build this functionality into a "Done" Product Increment during the Sprint.
- The Development Team does planning and design to make sure that they will achieve the Sprint Goal. May invite other people to attend to provide technical or domain advice.
- Scrum Master helps the team in case any arrangements need to be done for achieving the Sprint Goal.

SCRUM EVENT - DAILY SCRUM

- A **stand-up meeting** between the Scrum Team. It's a **15-minute time-box** daily event generally held at the same place and the same time.

- **Input ??** 3 questions –

"What did I do yesterday?", "What will I do today?" & "Are there any issues or Impediments?"

- Optimises team collaboration and performance by inspecting the work since last Daily Scrum and adapts its progress towards the Sprint Goal.

- Helps the team in terms of better communication, decision-making and improved level of knowledge.

SCRUM EVENT - SCRUM REVIEW

- A time-box event of **1 hr x num. of weeks of a Sprint** (incl. Product Owner & key stakeholders) held at the end of each Sprint.

- Hands-on with what's been built, and active discussion of :
  - **What** is the current state of the Product?
  - **What** should be added to the Product or improved in future Sprints to deliver maximum value?

- **Output: ??** Revised Product Backlog by Product Owner

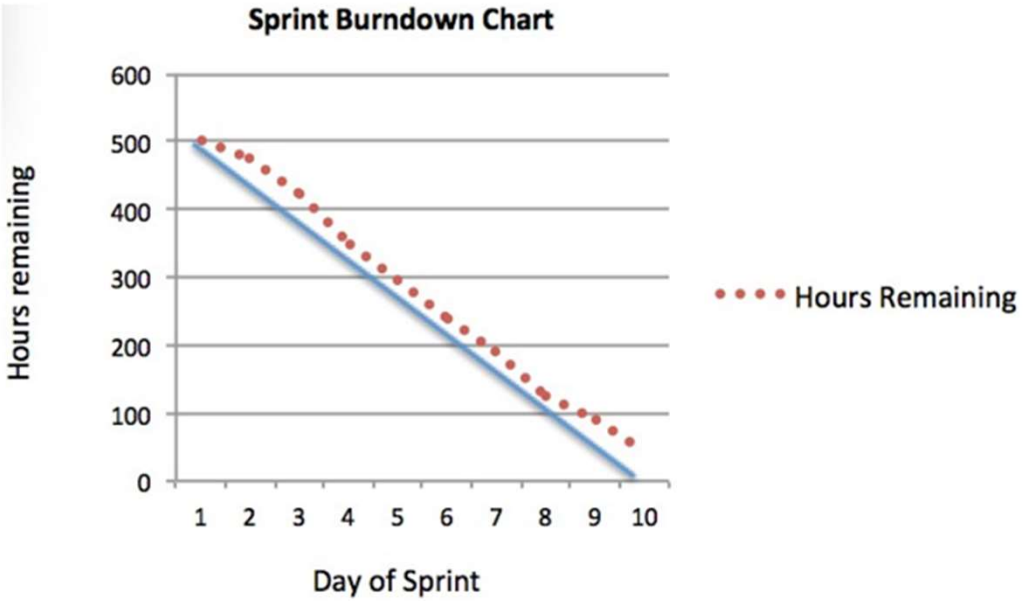
SCRUM EVENT - SCRUM RETROSPECTIVE

- A time-box event of **45 mins x num. of weeks of a Sprint** between the Scrum Team at the end of the sprint after Sprint Review.

- Active discussion of:
  - Inspect **how** the sprint went with regard to process, tool, and people.
  - Identify items that went well and potential improvements.
  - Create an action plan to implement improvements in the Scrum Team.

- **Input ??** Results from the Sprint
- **Output ??** Lesson learned, Improvements, and action list for the next Sprint.

SPRINT BURNDOWN CHART

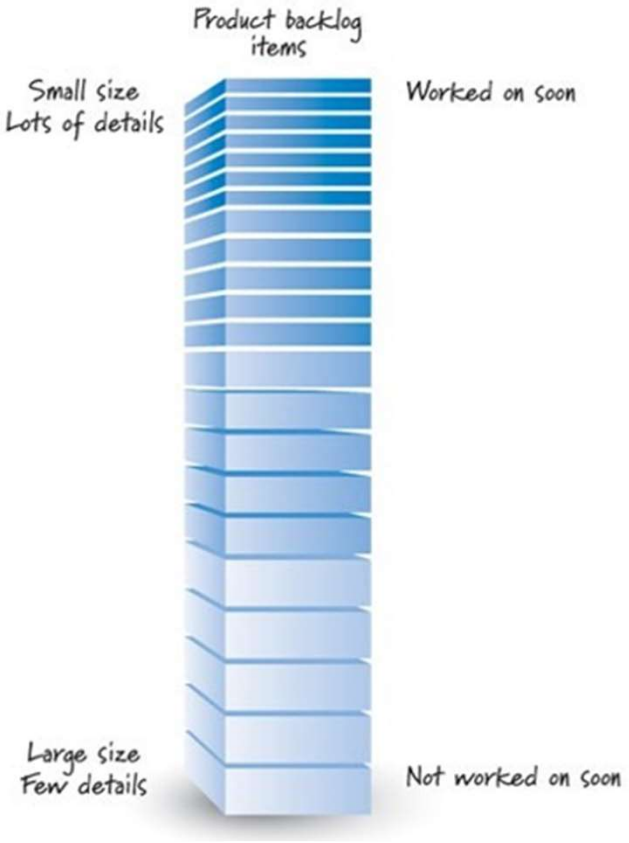


- A graphical representation of work left to do versus time.
- Purpose:
  - Monitoring the project scope creep
  - Keeping the team running on schedule
  - Comparing the planned work against the team progression

VELOCITY

- Velocity is the average size of Product Backlog items, in number of points, that the team can deliver during a Sprint.
- Measures the **amount of work done** during a Sprint. A key metric in Scrum.
- Evaluated only as an average. One Sprint's number is meaningless
- Tasks are estimated in real units using hours or quarter-days.
- Points cannot be converted to hours.

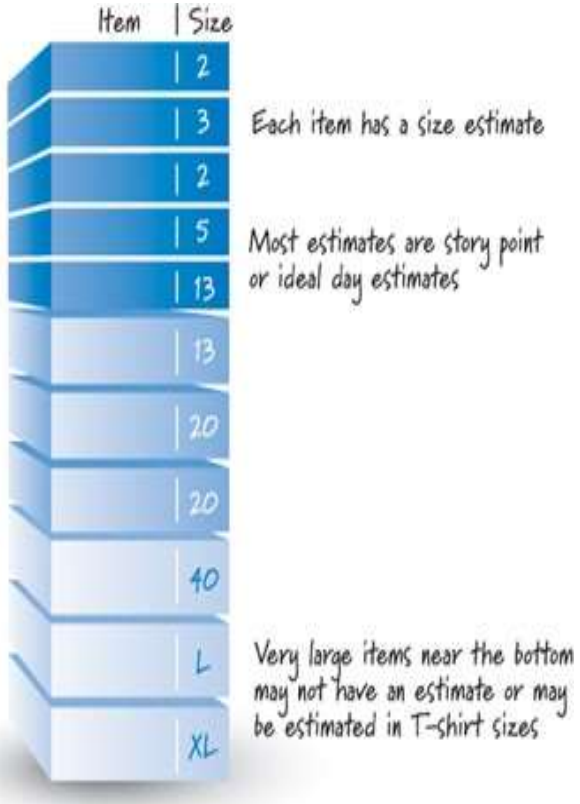
CHARACTERISTICS OF A GOOD PRODUCT BACKLOG



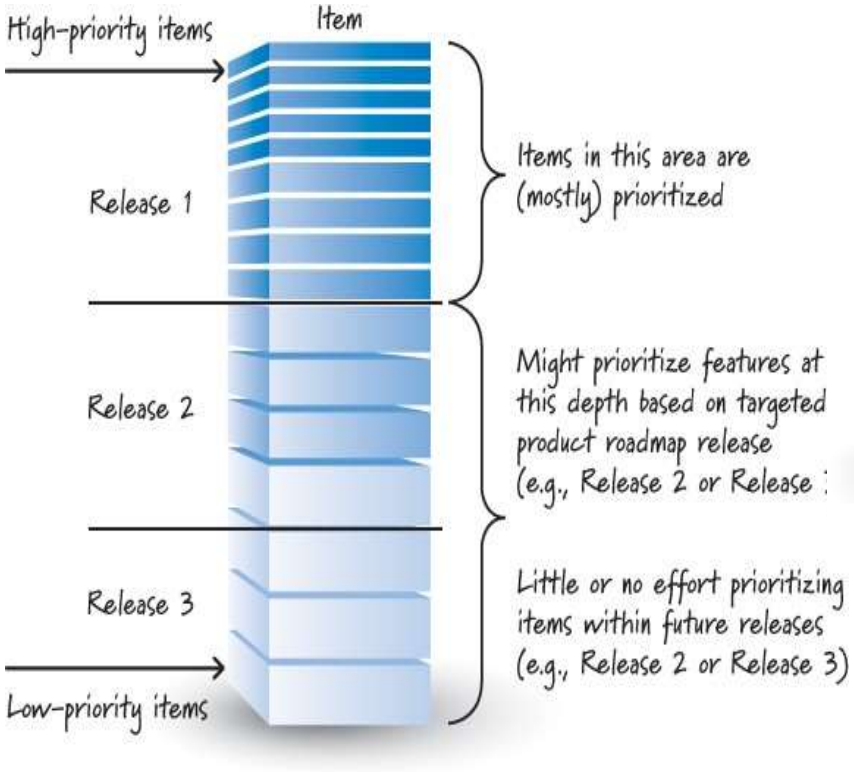
- **Detailed appropriately**
  - Not all items in a PB will be at the same level of detail at the same time.
  - PBIs that we plan work on soon should be at the top of the backlog, small in size and very detail so that they can be work on in the near sprint. BPIs that will be worked on later can be larger and less detail.
- **Emergent**
  - The product backlog is never complete or frozen. Instead, it is continuously updated based on a stream of economically valuable information that is constantly arriving.

CHARACTERISTICS OF A GOOD PRODUCT BACKLOG

- **Estimated**
  - Each PBI has a size estimate corresponding to the effort required to develop the item. PO uses these estimates as one of several inputs to help determine a PBI's priority (and therefore position) in the PB.



- **Prioritized**
  - Useful to prioritize the near-term items that are destined for the next few Sprints

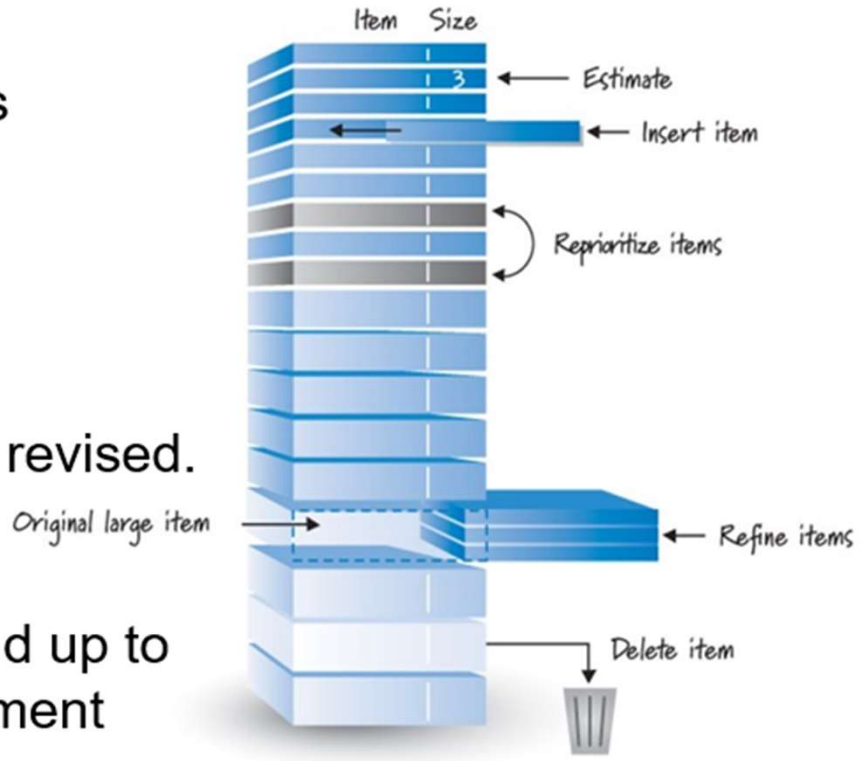


PRODUCT BACKLOG GROOMING (REFINEMENT)

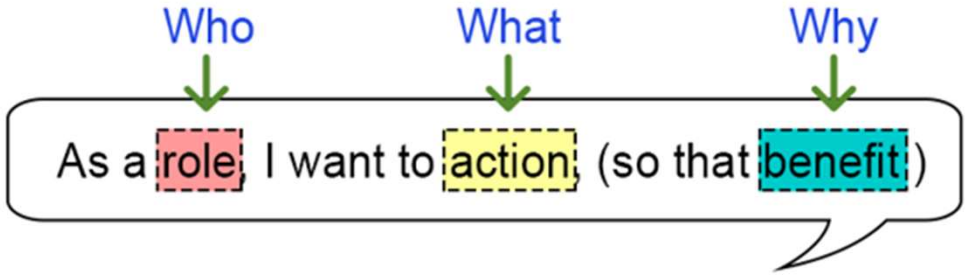
- Refers to a set of three principal activities:
  - Creating and refining (adding details to) PBIs
  - Estimating PBIs
  - Prioritizing PBIs

- An **ongoing** process - Items are reviewed and revised.

- As a general rule, the Development Team spend up to 10% of its time each sprint assisting with refinement activities.



USER STORIES



- User Stories are expressed in everyday language and describe a specific **goal** (what) from the **perspective of a user** (who) along with the reason (why) he/she wants it.

- **Epics-** a **large** body of work that can be broken down into multiple features. It may start out very big with limited detail.

- The Scrum Team splits these Epics into a smaller, more detailed User Stories in Product Backlog grooming/refinement.

3Cs Approach

Card

- represents the capture of the statement of intent of the user story.
- traditionally written on index cards or post-it notes.



Cards can be re-organized when agile requirements change.

3Cs Approach

Conversation

- details behind the User Stories ,where the clearer understanding comes from.
- provides shared context that cannot be achieved via formal documentation.

Confirmation

- represents acceptance criteria of the user story which confirms the intended value is delivered

WHAT IS ACCEPTANCE CRITERIA

- A set of predefined scope and requirements that must be executed by developers to consider the user story finished.
- May include:
  - **Functional Criteria:** Identify specific user functions or business processes that must be in place. A functional criterion might be "A user is able to access a list of available reports."
  - **Non-functional Criteria:** Identify specific non-functional conditions the implementation must meet, such as design elements. A non-functional criterion might be "Edit buttons and Workflow buttons comply with the Site Button Design."

CHARACTERISTICS OF A GOOD ACCEPTANCE CRITERIA

- State an intent not a solution (e.g. “The user can choose an account” rather than “The user can select the account from a drop-down”)

- Are independent of implementation (ideally the phrasing would be the same regardless is implemented on e.g. web, mobile or a voice activated system)

- Are relatively high level (not every detail needs to be in writing)

Teamwork in Scrum

- The ‘Scrum master’ is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside of the team.
- The whole team attends short daily meetings (Scrums) where all team members share information, describe their progress since the last meeting, problems that have arisen and what is planned for the following day.
  - This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them.

Scrum benefits

- The product is broken down into a set of manageable and understandable chunks.
- Unstable requirements do not hold up progress.
- The whole team have visibility of everything and consequently team communication is improved.
- Customers see on-time delivery of increments and gain feedback on how the product works.
- Trust between customers and developers is established and a positive culture is created in which everyone expects the project to succeed.

Key points

- Agile methods are incremental development methods that focus on rapid software development, frequent releases of the software, reducing process overheads by minimizing documentation and producing high-quality code.
- Agile development practices include
  - User stories for system specification
  - Frequent releases of the software,
  - Continuous software improvement
  - Test-first development
  - Customer participation in the development team.

Key points

- ✧ Scrum is an agile method that provides a project management framework.
  - It is centred round a set of sprints, which are fixed time periods when a system increment is developed.
- ✧ Many practical development methods are a mixture of plan-based and agile development.
- ✧ Scaling agile methods for large systems is difficult.
  - Large systems need up-front design and some documentation and organizational practice may conflict with the informality of agile approaches.